

Task 1: A.

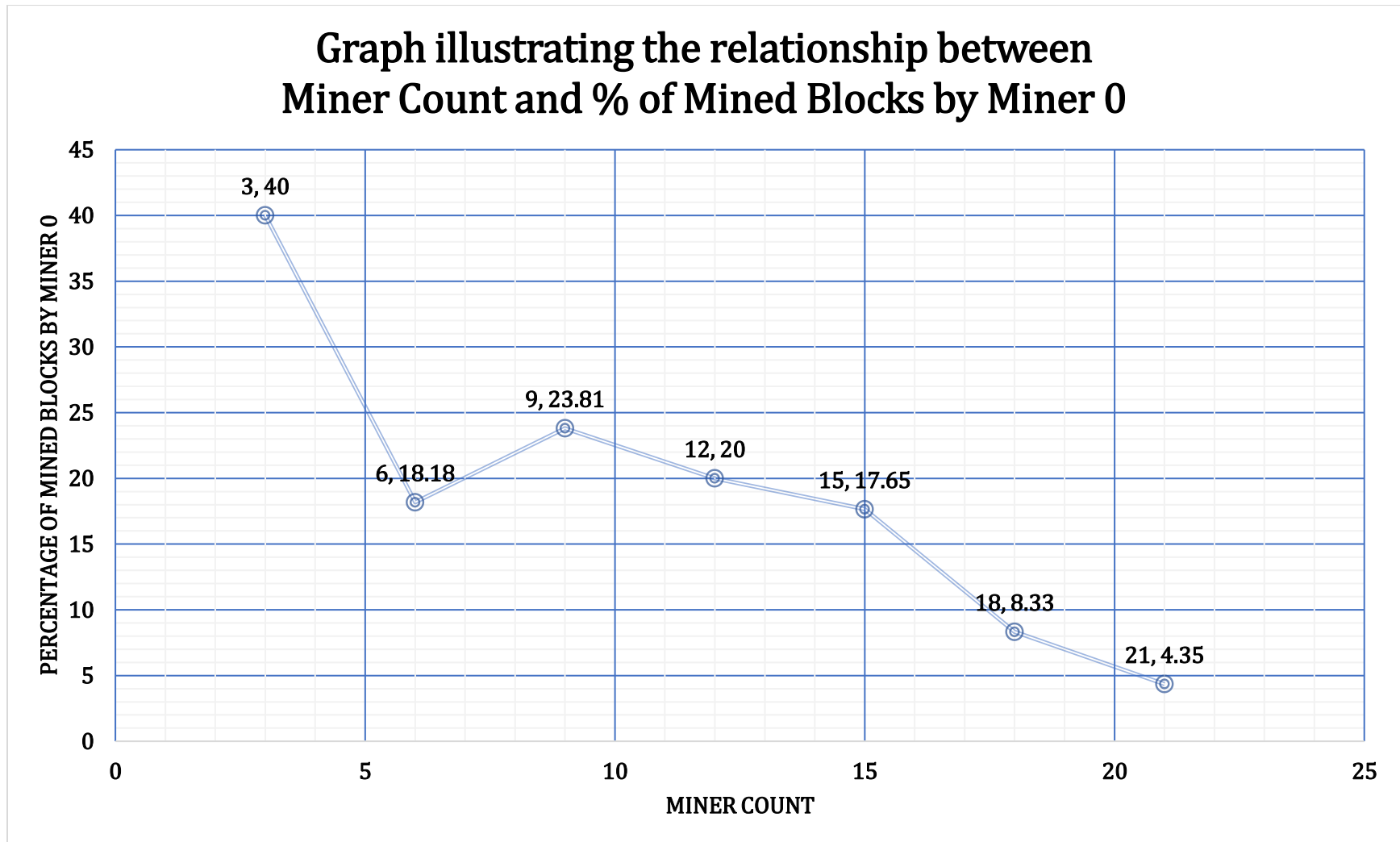


Figure 1 Graph illustrating the relationship between Miner Count and % of Mined Blocks by Miner 0

An analysis of the empirical data presented in Figure 1 reveals a pronounced inverse relationship between the total network miner count and the percentage of blocks successfully mined by Miner 0. As the parameter for the total number of nodes in the network increases from 3 to 21, Miner 0's success rate depreciates significantly from 40% down to 4.35%. Though there was a slight rise from 18.18% to 23.81% when the miner count increased from 6 to 9, but after 9, further increase in the number of miners kept decreasing the percentage of blocks mined by Miner 0.

In a Proof-of-Work (PoW) distributed ledger, the probability of a specific node appending the next block is directly proportional to its relative hash rate compared to the aggregate network hash power. Assuming Miner 0's absolute computational output remains static, the introduction of additional miners inherently dilutes Miner 0's relative percentage of the total network hash rate. Consequently, heightened computational competition directly suppresses the individual probability of block discovery.

Task 1: B.

If the total miner count and aggregate network difficulty remain constant over an extended temporal horizon, the profitability of an individual miner will statistically converge to a steady-state value. While the stochastic nature of the cryptographic hashing process (modeled as a Poisson process) introduces short-term variance in block discovery, the Law of Large Numbers dictates that the miner's long-term block yield will mirror their proportional hash rate. Therefore, their expected gross profit will stabilize at a constant rate, mathematically defined by their fixed probability of capturing the block reward and associated transaction fees, minus their continuous operational expenditures (e.g., electricity and hardware depreciation). In other words, the actual profit will fluctuate from run to run because mining is probabilistic. A miner may occasionally mine more or fewer blocks than expected, causing its profit to increase or decrease. Over a sufficiently long period, the average profit should become more stable.

Task 2: A.

The graphical data illustrated in Figure 2 below demonstrates a critical, non-linear inverse correlation between the block creation interval ($B_{interval}$) and the occurrence of stale blocks. At highly compressed intervals - specifically 10 and 20 seconds - the network exhibits notable stale rates of 2.22 and 1.03, respectively. However, as the $B_{interval}$ parameter is extended to 40 seconds and beyond, the stale rate precipitously drops to absolute zero.

**Graph illustrating the relationship between
Block Interval (Binterval) and Stale Rate**

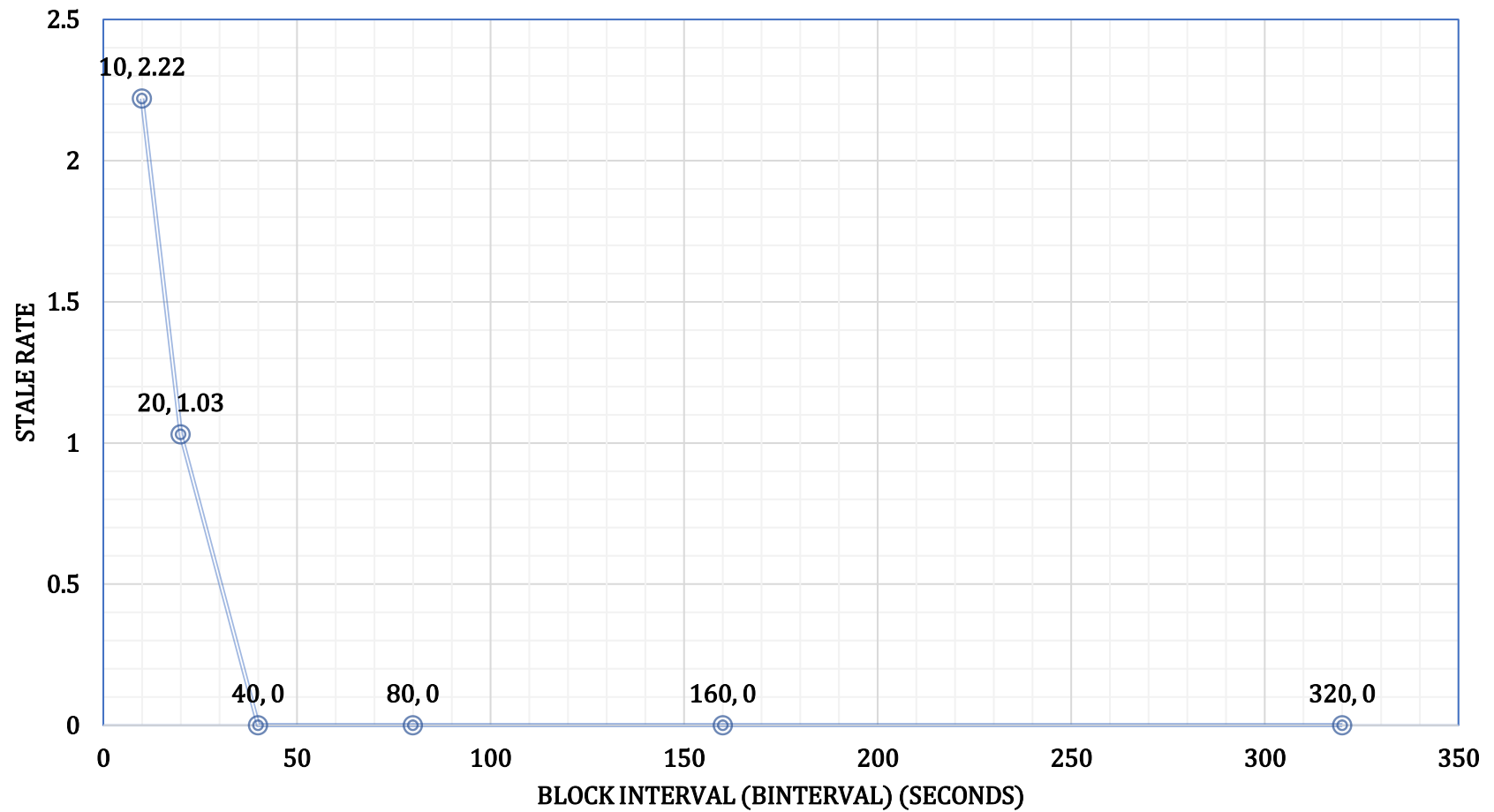


Figure 2 Graph illustrating the relationship between Block Interval (Binterval) and Stale Rate

Task 2: B.

A stale (or orphaned) block is generated when two or more miners solve the cryptographic puzzle nearly simultaneously. At lower Binterval thresholds, the temporal window required to propagate a newly discovered block across the global peer-to-peer network is roughly equivalent to, or greater than, the time it takes to mine a subsequent block. This propagation latency induces localized chain forks, as disparate nodes continue to expend computational energy mining atop outdated chain states. Conversely, expanding the Binterval provides ample latency for complete global block propagation. This ensures the entire network synchronizes and achieves consensus on the canonical chain state before the subsequent block discovery cycle is finalized, thereby eliminating the generation of competing, stale blocks. In conclusion, referring to the phenomenon observed in Figure 2, as Binterval increases, the stale rate decreases as a shorter block interval causes blocks to be generated more frequently, increasing the probability of competing blocks and stale blocks.

Task 3: A.

To implement a dynamic selection mechanism for transaction processing, the execution logic within the Transaction.py module was structurally augmented. A class-level configuration variable, designated as ORDERING_TOGGLE, was instantiated within the LightTransaction class. This control variable governs the conditional flow of the execute_transactions function, effectively bypassing or engaging the default profit-maximizing miner strategy based on the defined state.

The modification introduces the following conditional logic block to the simulator:

```
if LightTransaction.ORDERING_TOGGLE == "FIFO":
    print("Executing block with FIFO ordering")
else:
    print("Executing block with FEE-priority ordering")
    pool = sorted(pool, key=lambda x: x.fee, reverse=True)
```

Fee-Priority Execution (Default Mode): As explicitly documented in Figure 3, when the ORDERING_TOGGLE variable is initialized to "FEE", the simulator routes execution through the else conditional branch. In this state, the system enforces the default rational miner protocol by employing Python's built-in sorted() function. By applying a lambda function (key=lambda x: x.fee, reverse=True), the algorithm deterministically rearranges the pool array, sorting the pending mempool transactions in strictly

descending order based on their attached cryptographic fee. The terminal output interface continuously verifies this active operational state by printing the diagnostic string "Executing block with FEE-priority ordering" upon the instantiation of each subsequent block.

[illegible]

Figure 3 Executing Block with FEE-priority Ordering

FIFO Execution Mode: Conversely, when the ORDERING_TOGGLE is configured to "FIFO", the script actively bypasses the sorting algorithm entirely. The system systematically ingests and processes the transaction pool array in its native, temporally

sequential arrival order (simulated by the initial array shuffle). The terminal execution subsequently confirms this algorithmic override by continuously outputting the diagnostic string "Executing block with FIFO ordering".

The screenshot displays a Python IDE with the following components:

- EXPLORER:** A file tree on the left showing a project structure with files like `Block.py`, `Consensus.py`, `Transaction.py`, and others. `Transaction.py` is selected.
- Model:** A breadcrumb trail at the top of the editor showing the path: `Models > Transaction.py > LightTransaction > create_transactions`.
- Editor:** The main code area showing the implementation of the `LightTransaction` class and the `execute_transactions` function. The code includes logic for creating a pool of transactions, shuffling them, and selecting them based on a gas limit and ordering toggle.
- TERMINAL:** A terminal window on the right showing the output of the program. It displays a continuous stream of the message "Executing block with FIFO ordering", confirming that the FIFO ordering algorithm is being used.

```

class LightTransaction():
    pending_transactions=[] # shared pool of pending transactions
    ORDERING_TOGGLE = "FIFO"
    def create_transactions():
        LightTransaction.pending_transactions=[]
        pool= LightTransaction.pending_transactions
        Psize= int(p.Tn * p.Binterval)
        for i in range(Psize):
            tx= Transaction()
            tx.id= random.randrange(100000000000)
            tx.sender = random.choice (p.NODES).id
            tx.to= random.choice (p.NODES).id
            tx.size= random.expovariate(1/p.Tsize)
            tx.fee= random.expovariate(1/p.Tfee)
            pool += [tx]
        random.shuffle(pool)

    ##### Select and execute a number of transactions to be added in the next block #####
    def execute_transactions():
        transactions= [] # prepare a list of transactions to be included in the block
        size = 0 # calculate the total block gaslimit
        count=0
        blocksize = p.Bsize
        pool= LightTransaction.pending_transactions

        # pool = sorted(pool, key=lambda x: x.fee, reverse=True) # sort pending transactions in the pool based on the gasPrice value
        if LightTransaction.ORDERING_TOGGLE == "FIFO":
            print("Executing block with FIFO ordering")
        else:
            print("Executing block with FEE-priority ordering")
            pool = sorted(pool, key=lambda x: x.fee, reverse=True)

        while count < len(pool):
            if (blocksize >= pool[count].size):
                blocksize -= pool[count].size
                transactions += [pool[count]]
                size += pool[count].size
                count+=1

        return transactions, size

class FullTransaction():
    def create_transactions():

```

Figure 4 Executing Block with FIFO Ordering

Task 3: B.

The architectural decision between First-In-First-Out (FIFO) and Fee-Priority transaction ordering carries profound implications for blockchain mechanics:

- **FIFO Ordering:**
 - *Pros (Fairness):* Implements a strictly egalitarian processing paradigm. Users are serviced based on temporal precedence rather than capital expenditure, democratizing network access and mitigating the systematic exclusion of economically disadvantaged participants during periods of high network congestion.
 - *Cons (Incentives & Security):* Fundamentally deteriorates miner economic incentives. By decoupling execution priority from financial remuneration, miners are deprived of a critical revenue stream. Furthermore, the network becomes highly vulnerable to Denial-of-Service (DoS) and spam vectors; malicious actors can easily saturate the mempool with zero-fee micro-transactions without incurring financial penalties, degrading network throughput.
- **Fee-Priority Ordering (Default):**
 - *Pros (Incentives & Security):* Establishes a robust, rational fee market. Miners maximize utility and profitability, incentivizing the deployment of extensive computational resources to secure the network. It inherently deters network spam by imposing a dynamic financial cost on block space consumption.
 - *Cons (Fairness):* Introduces economic friction and marginalization. During transaction surges, network fees escalate dynamically, effectively pricing out lower-tier economic actors and inherently favoring institutional or high-net-worth capital transfers.

To conclude, FIFO provides greater fairness to users, while fee-priority ordering provides stronger incentives for miners to maximize transaction-fee revenue.

Task 4: A.

Simulating Bitcoin's disinflationary supply schedule requires dynamically adjusting the block reward based on the chain's accumulated depth. The programmatic integration necessitates an exponential decay function, which can be mathematically modeled as:

$$R_{current} = \frac{R_{base}}{2^{\lfloor \frac{D_{current}}{I_{halving}} \rfloor}}$$

Where R_{base} is the initial block reward, $D_{current}$ is the current block depth, and $I_{halving}$ represents the halving interval parameter (“The Mathematics of Bitcoin Halvings: Cracking the Equations and Formulas,” 2024).

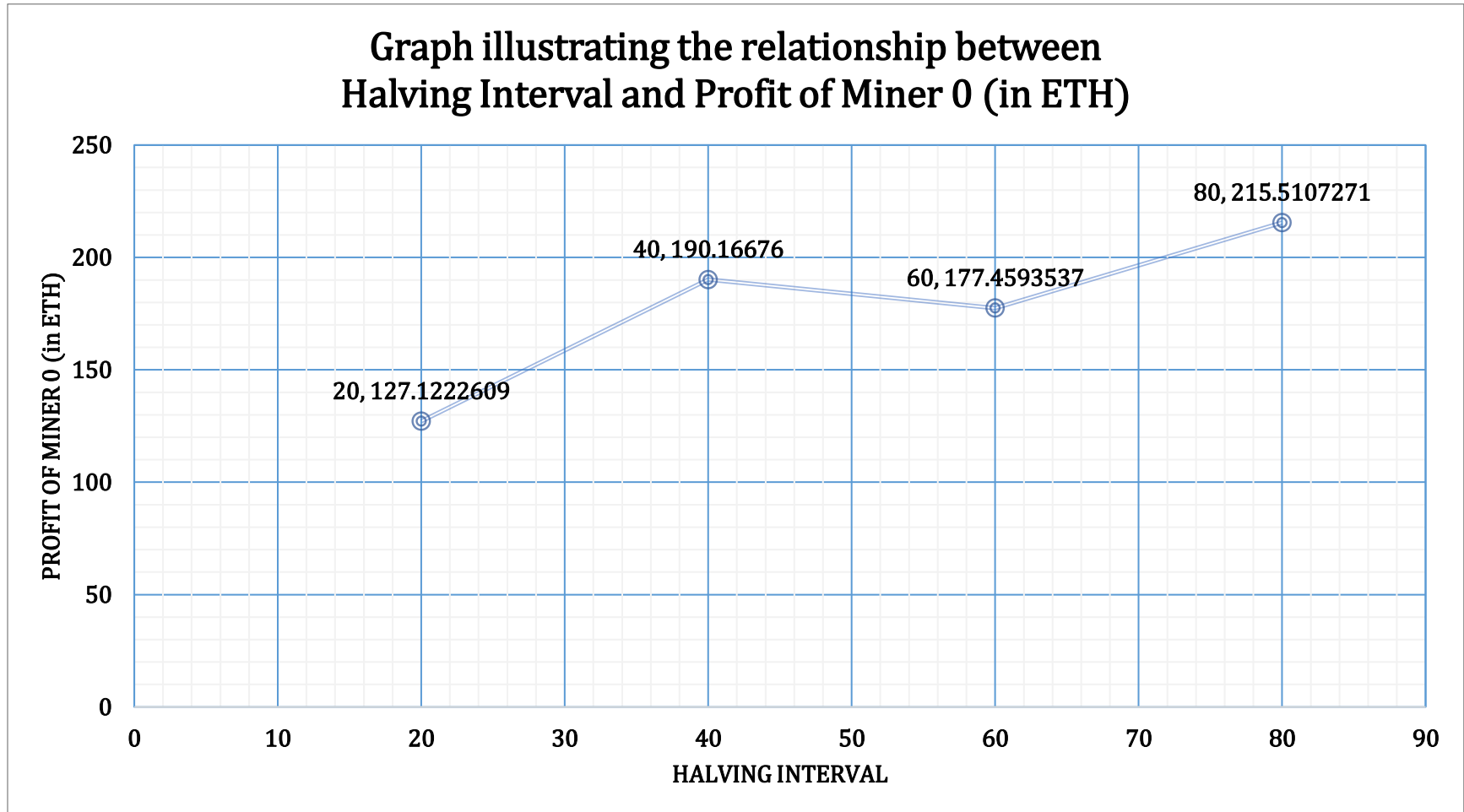


Figure 5 Graph illustrating the relationship between Halving Interval and Profit of Miner 0 (in ETH)

The simulation results in Figure 5 chart the total profit of Miner 0 across scaling halving intervals (20, 40, 60, and 80 blocks). The data indicates an overarching positive correlation: as the interval stretches (delaying the reward depreciation), Miner 0 accumulates a higher volume of the native asset (e.g., scaling from 127.1222609 ETH at an interval of 20, up to 215.5107271 ETH at an interval of 80). However, while moving from halving interval of 40 to 60, there was a slight decrease of the profit of Miner 0 from 190.16676 ETH to 177.4593537 ETH, but overall from 20 to 60, there was an increase as stated before.

Task 4: B.

Expanding the halving parameter sequentially delays the reward depreciation. At an interval of 40, Miner 0's profit recovers to 190.16676 ETH (with a minor variance at the 60 interval yielding 177.4593537 ETH, likely due to the stochastic nature of block discovery). Ultimately, extending the interval to the maximum tested threshold of 80 drastically reduces the total number of halving events. This allows the miner to capitalize on the higher base reward for a substantially prolonged duration, maximizing Miner 0's total yield to 215.5107271 ETH.

As the number of halving epochs increases, the deterministic block reward awarded to a miner depreciates exponentially. The empirical data extracted from the simulation and illustrated in the halving interval graph (Figure 5) explicitly corroborates this mathematical reality. Within the fixed temporal bounds of the simulation, a highly compressed halving interval of 20 induces a maximum frequency of halving events. Consequently, the block reward degrades rapidly, severely suppressing Miner 0's aggregate profit to a simulation low of 127.1222609 ETH.

Dictated by the protocol's asymptotic monetary policy, the block subsidy will continuously halve until it diminishes to a fractional value that mathematically rounds down to zero. Once this terminal state is reached, the network entirely ceases the cryptographic minting of new assets. To sustain decentralized operations, subsidize computational overhead (e.g., hardware depreciation and electrical expenditure), and maintain network security, miner revenue must transition to being derived entirely from transaction fees.

The Necessity of Fee-Priority Ordering

In a zero-block-reward paradigm, a Fee-Priority transaction ordering model transcends being a mere optimization and becomes an absolute cryptographic and economic necessity.

- **The Failure of FIFO:** As observed in the Experiment 3 modifications, a First-In-First-Out (FIFO) architecture forces nodes to process mempool transactions strictly based on temporal arrival, completely disregarding the attached financial fee. If a

blockchain were to operate on a FIFO model in the absence of a block subsidy, miners would be computationally compelled to process zero-fee or low-value transactions without any intrinsic financial yield. This structural flaw would obliterate the economic incentive to mine, triggering a catastrophic capitulation of the network hash rate and rendering the distributed ledger highly vulnerable to 51% attacks (Team, 2025).

- **The Security of Fee-Priority:** Alternatively, the Fee-Priority framework empowers miners to systematically prioritize the highest-paying transactions. This inherently cultivates a dynamic, competitive fee market where network participants continuously bid for limited block space. This market-driven mechanism guarantees that, even in the permanent absence of a block reward, miners receive continuous and adequate financial compensation. This ensures the perpetual sustainability of the distributed consensus mechanism and overall network integrity.

References

Team, I. (2025, August 21). *What is a 51% Attack on Blockchain? Risks, Examples, and Costs Explained*. Investopedia.

<https://www.investopedia.com/terms/1/51-attack.asp>

The mathematics of Bitcoin halvings: Cracking the equations and formulas. (2024, April 18). *TradingView*.

<https://www.tradingview.com/news/cointelegraph:8f5370910094b:0-the-mathematics-of-bitcoin-halvings-cracking-the-equations-and-formulas/>